

Alberta Buck – Worked Examples

Perry Kundert

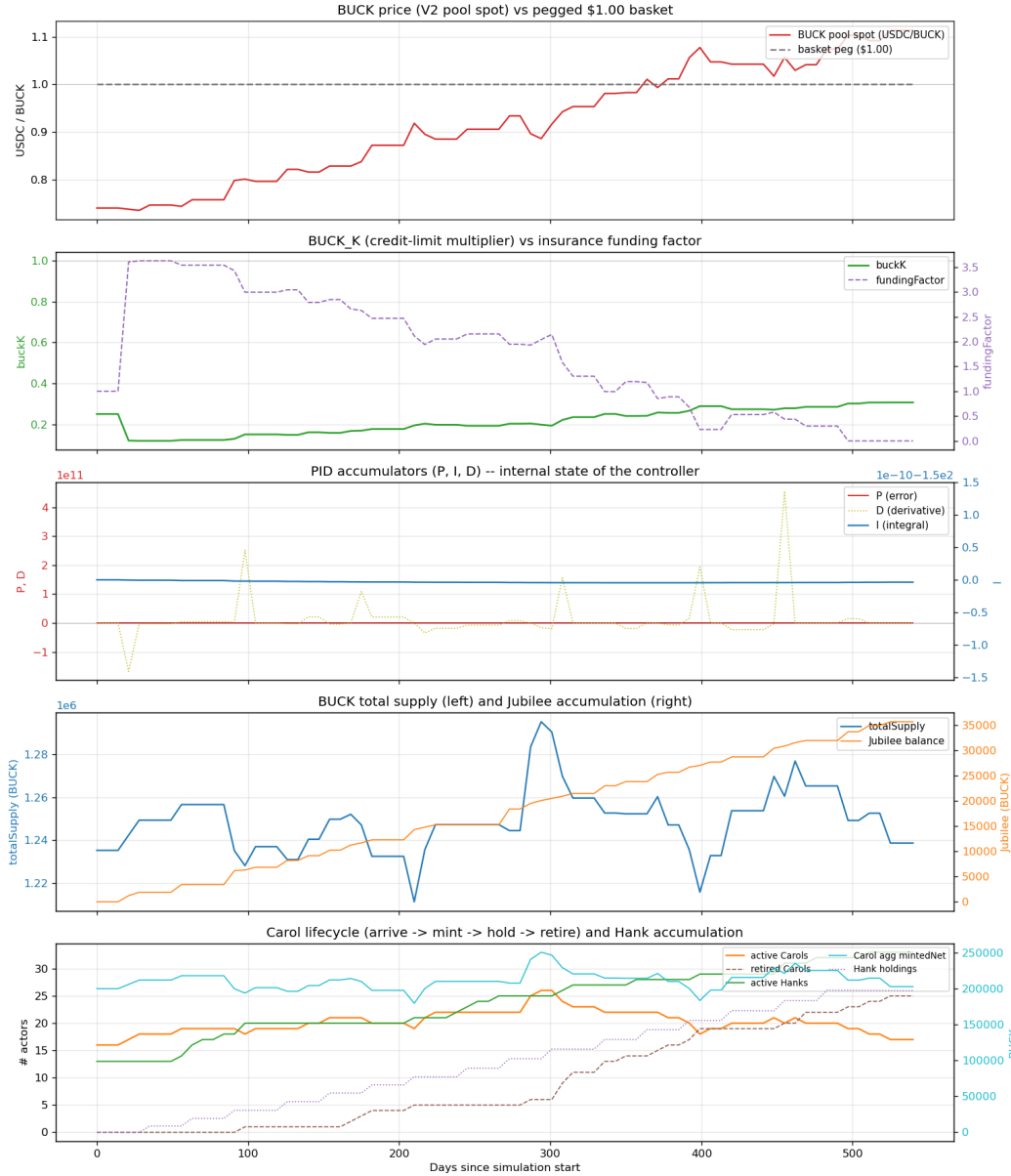
2026-05-10

1 Bootstrap, Funding Factor, and the BUCK_K Feedback Loop

The first worked example traces how Alberta Buck reaches monetary equilibrium from a cold start. Two mechanisms operate in tandem:

- The **insurance funding factor**, a counter-cyclical constraint on minting that creates pre-mint demand for BUCK proportional to how undervalued BUCK currently trades.
- The **BUCK_K credit-limit feedback**, where the PID’s `buckK` multiplier expands or contracts every holder’s borrowing capacity, voluntarily inducing burns during inflation and mints during deflation.

The shipped scenario test (`test/BuckEquilibriumScenario.t.sol`) demonstrates both loops against a real `Buck.sol` + `BuckCredit.sol` stack and a Uniswap V2 BUCK/USDC pool. The basket is pegged to \$1.00 USDC for clarity (the live configuration tracks a 4-pool commodity basket; the dynamics are identical).



1.1 Insurance Funding Factor

When a holder calls `Buck.mint(amount)`, the per-NFT allocator computes a `poolPrincipal` amount that's the insurance contribution credited to the mutual-coverage pool. Before the mint is accepted, the holder must already hold

```
required = poolPrincipal * fundingFactor / 1e18
```

of BUCK in their wallet. The pre-balance is **not consumed** – it's skin-in-the-game collateral that must remain held going forward.

The factor itself is counter-cyclical in BUCK's deviation from basket:

```
factor = max(0, 1 + 10 * (basket - BUCK) / basket)
```

BUCK price (basket = \$1.00)	Funding factor	Interpretation
\$1.10 (10 % deflation)	0.0	gate fully opens
\$1.05 (5 % deflation)	0.5	50 % of premium
\$1.00 (parity)	1.0	full premium pre-held
\$0.97 (3 % inflation)	1.3	30 % surcharge
\$0.95 (5 % inflation)	1.5	50 % surcharge
\$0.90 (10 % inflation)	2.0	2x surcharge
\$0.80 (20 % inflation)	3.0	3x surcharge

The 10:1 slope is hardcoded in `BuckKController.fundingFactor()`; later governance versions may parametrise it. `BuckKControllerStatic` returns 0, which is the convention for "gate disabled" (Phase-0 environments without oracles).

1.1.1 Bootstrap exemption

At `totalSupply = 0` no BUCK exists, so any non-zero requirement is unsatisfiable. `Buck._mintAllocated` explicitly bypasses the gate in that window so the first mint can seed the system. Once any BUCK has been minted, every subsequent mint must satisfy the live factor.

1.1.2 Why this matters for the bootstrap

Picture the system on day 0: a single `BuckCredit` NFT has been issued, no `Buck` has been minted, the BUCK/USDC pool is empty. Whoever mints first (the bootstrap participant) seeds the supply; their first action is usually to provide that BUCK as liquidity in the BUCK/USDC pair so others have a market to interact with.

After that point, every newcomer faces the funding factor. When the pool has just been seeded – BUCK trading near \$1.00 – factor is roughly 1.0: each new minter must accumulate the full insurance principal as BUCK **before** their first mint. That accumulation is itself a BUCK buy from the pool, supporting the BUCK price.

As more newcomers arrive and mint+sell, BUCK drifts below parity. Factor rises. The next newcomer must accumulate **more** BUCK pre-mint than the previous one. The barrier-of-entry mechanically supports BUCK demand proportional to how undervalued BUCK has become.

This is the "running start" property: the system cannot mint itself into oblivion because every new mint is preceded by BUCK demand whose magnitude scales with the very inflation pressure it would otherwise cause.

1.2 BUCK_K Credit-Limit Feedback

In parallel with the funding factor, the on-chain PID controller adjusts `buckK`, the credit-limit multiplier. Each holder's effective borrowing capacity is

```
limit = totalCreditValue * buckK / 1e18
```

The shipped controller defines the PID error as

```
error = buckPrice - basketCost ; process minus setpoint
```

so that under canonical *positive* PID gains:

- **Inflation** (BUCK trading below basket): `error < 0` drives `buckK` *down*. Existing holders' effective limits contract. Holders who target a fixed utilization fraction (e.g. 50 % of limit) must burn BUCK to stay within the new, lower limit.
- **Deflation** (BUCK trading above basket): `error > 0` drives `buckK` *up*. Holders gain headroom and can mint more BUCK to lock in the premium, selling into the pool and pushing spot back down toward basket.

This sign convention is enforced by two canary tests in `test/BuckKController.t.sol`:

- `test_sign_convention_inflation_contracts_credit` – BUCK at \$0.95 must drive `buckK` *below* 1.0 within one cycle.
- `test_sign_convention_deflation_expands_credit` – BUCK at \$1.05 must drive `buckK` *above* 1.0 within one cycle.

Both tests rely on positive gains; negating any gain inverts the controller and the canary fails immediately. Any production deployment that selects gains **MUST** pick positive values under this convention.

1.3 Carol/Hank/Alice dynamics

The scenario test populates the simulation with three actor archetypes:

- **Alice** (singular, identified): the bootstrap participant. Holds the initial \$4M face-value `BuckCredit`, mints \$1M BUCK at the genesis bypass, and pairs it 1:1 with USDC in the V2 pool to provide market liquidity.
- **Carol[1..50]** (synthetic public identities): borrowers. Each holds a \$100K face-value vehicle-style credit (1.5%/yr premium) and runs a full lifecycle:
 1. Arrives at a Gaussian-distributed time (peak day 180, std 100 days).
 2. Pre-buys `poolPrincipal * fundingFactor / 1e18` BUCK from the pool.
 3. Mints 50% of her current credit limit; immediately swaps the proceeds to USDC.
 4. Holds the position for ~270 days (Gaussian-distributed).
 5. Departs by buying her `mintedNet` BUCK back from the pool and burning it (releases coverage, returns the insurance pool principal).

A small fraction of active Carols re-evaluate monthly to track `buckK`: if `buckK` has risen and BUCK is trading at a premium they top up (mint more, dump for premium); if `buckK` has fallen and BUCK trades at a discount they reduce (buy back, burn).

- **Hank[1..40]** (synthetic public identities): opportunistic holders. Each arrives at a Gaussian-distributed time and spends \$500 USDC per month on BUCK, building a long-term position. Hanks are price-aware: they skip months when BUCK trades above \$1.05 (no premium accumulation). They never burn; they're modelling demand for BUCK-denominated multi-year savings, inheritance Notes, and farmland.

Without Hanks the Carol-only market is one-sided: every Carol mints+dumps and the pool drifts permanently BUCK-heavy. With Hanks the demand side absorbs Carol supply at near-parity, letting the system find a real equilibrium where Carol-arrival flow = Carol-departure flow + Hank-buy flow.

1.4 Historical analogues

The funding-factor mechanism has direct precedents:

- **Initial margin in futures and equity margin trading:** opening a leveraged position requires posting cash collateral as a percentage of notional. Higher exchange-mandated initial-margin requirements (often imposed during volatile or stressed periods) function as a counter-cyclical brake on credit expansion – the same pattern as raising the funding factor when BUCK trades below basket.
- **Reserve requirements in fractional-reserve banking:** central banks historically required commercial banks to hold a percentage of deposits as reserves. Raising reserve requirements tightens credit creation capacity; lowering them loosens it. The funding factor implements the same principle at the level of an individual minter rather than an institution.
- **Gesell’s stamp scrip and demurrage currencies:** Silvio Gesell’s *Freigeld* proposal required holders to affix monthly stamps to bank notes to keep them valid. The stamp fee created continuous demand for the underlying currency (you had to buy stamps with the same currency you were stamping). The funding factor pre-mint requirement creates analogous demand: to mint BUCK you must first acquire BUCK.
- **Mutual-aid society buy-ins:** English friendly societies, modern benevolent mutuals, and cooperative banks have historically required new members to pay an initial contribution before drawing benefits. The funding factor formalises this: the new BUCK minter is by construction a contributor to the mutual-insurance pool, and they must demonstrate that contribution by holding BUCK already.
- **Tax-driven currency demand:** Modern Monetary Theory’s chartalism holds that fiat currency derives value from being the only acceptable medium for tax payment. The funding factor creates a similar structural demand sink: anyone who wants to use BUCK credit must first hold BUCK, and the magnitude of that requirement scales with the system’s monetary stress.
- **Cryptocurrency bootstrap auctions:** early-stage crypto networks often gate participation on pre-purchased tokens (pre-sale, initial DEX offering, vesting cliffs). The funding factor accomplishes the same goal – ensuring new entrants are economically aligned – without a one-shot pre-sale. The "entry fee" is paid in BUCK and remains in the holder’s wallet, so it is collateral rather than burn.

1.5 Reproducing this example

```
forge test --match-contract BuckEquilibriumScenarioTest -vv
python -m pytest alberta_buck/test/test_equilibrium_plot.py -v -s
```

The Forge test emits `test/vectors/equilibrium-scenario.json` (weekly snapshots over 18 simulated months). The Python script reads that JSON and writes `images/equilibrium-scenario.png` – the four-panel chart above.

2 Vehicle Insurance Lifecycle

This example traces a single year in the life of a vehicle-insurance BUCK_CREDIT: from collateral creation through AMM liquidity provision and swap-fee collection to final burn. Every figure in this section is generated from an actual Foundry test run – see `test/BuckLifecycle.t.sol` for the on-chain numbers and `alberta_buck/test/test_lifecycle_plot.py` for the visualisation.

2.1 Generating the data

```
forge test --match-test test_lifecycle -vv
```

This writes `test/vectors/lifecycle.json`. Run once; re-run whenever the contracts change. To regenerate the plot:

```
python -m pytest alberta_buck/test/test_lifecycle_plot.py -v -s
```

2.2 Scenario

Alice owns a \$10,000 vehicle. Her insurer issues a BUCK_CREDIT NFT with a 15 %/yr declining-balance depreciation schedule and a \$500 salvage floor (5 %). Alice activates the full face value and mints 5,000 BUCK against it.

Parameter	Value
Face value	\$10,000
Depreciation type	DECLINING_BALANCE
Annual rate	15 % (1500 bp)
Salvage floor	\$500 (5 %)
Annual premium rate	1.5 % (150 bp)
BUCK minted	5,000 BUCK
Pool seed	5,000 BUCK + 5,000 USDC
Bob's monthly swaps	$12 \times 500 \text{ USDC} \rightarrow \text{BUCK}$
Pool fee	0.3 % (Uniswap V2)

The mint draws cheapest-first coverage. At a 1.5 % premium and $10\times$ pool-ROI inverse, the per-BUCK denom is

$$\text{denom} = 10000 - 150 \times 10 = 8500$$

`so=take = 5000 × 10000 / 8500 = 5882 BUCK= of coverage is drawn and ~=poolPrincipal = 882 BUCK= is minted to insurancePool1. Total supply after mint: $5882.`

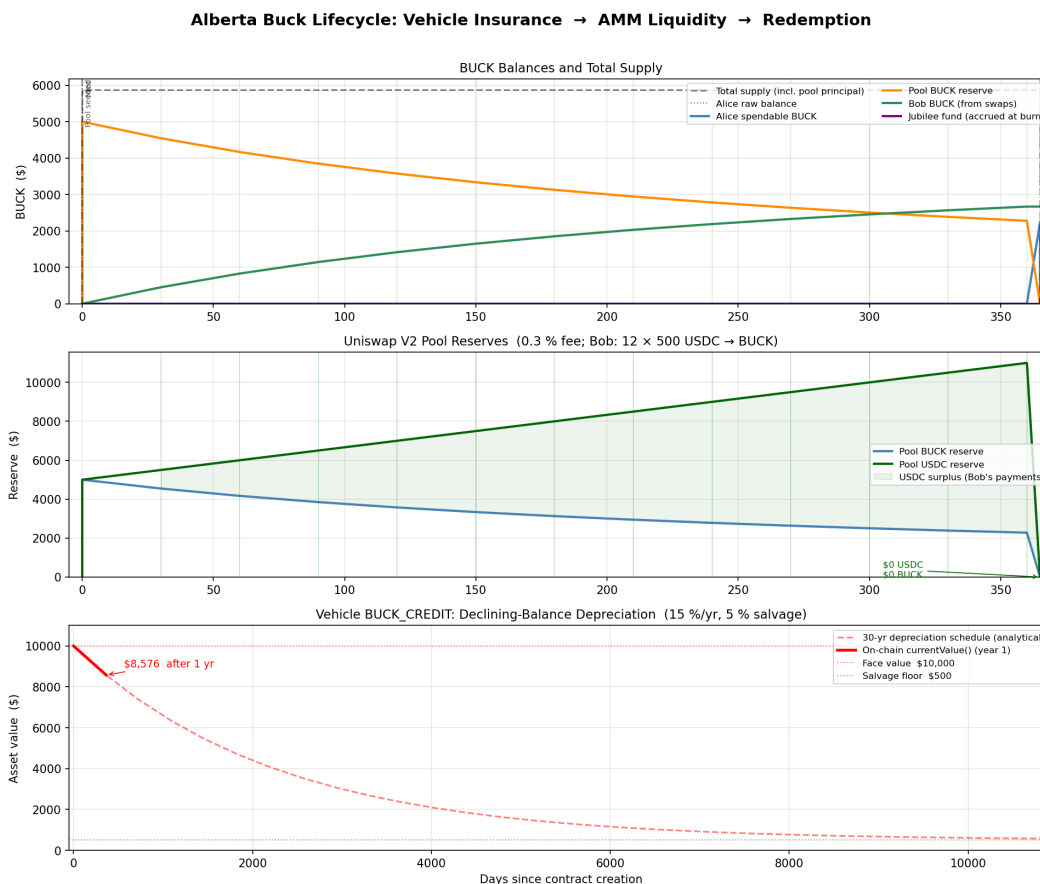
Alice deposits all 5,000 BUCK plus 5,000 USDC into a Uniswap V2 pair (1:1 ratio). Bob then buys 500 USDC of BUCK each month for a year.

2.3 Key outcomes after one year

Event	BUCK balance	USDC balance	Notes
Mint	5,000	10,000	882 BUCK principal to pool
Pool seed	0	5,000	All BUCK deposited
After swap 12	0	5,000	5000 USDC→BUCK paid in by Bob
Liquidity removed	2,232	16,000	Pool rebalanced; USDC up \$6K
After burn	0	16,000	Coverage unwound; jubilee \$117

Alice exits with \$16,000 USDC – a \$6,000 gain from Bob’s swap fees – having paid zero demurrage herself (her BUCK sat in a Carrying pool the whole time, pre-accruing Jubilee instead). The vehicle credit shows \$8,576 current value after one year of 15 % declining-balance depreciation.

2.4 Lifecycle chart



The chart has three panels:

- **BUCK balances (top):** Alice’s raw and spendable BUCK track identically until liquidity removal, when the pool’s carried `buckSeconds` arrive with the returned tokens (raw > spendable by the \$46 carried fee). Bob’s BUCK accumulates linearly as he buys each month. The total supply stays flat at \$5,882 until Alice burns, whereupon it drops to \$3,256 (the pool principal stays because `insurancePool` was only partially wound back against Alice’s partial burn). The Jubilee spike at burn day reflects the 2 %/yr accrual on the full supply over the year.
- **Pool reserves (middle):** Uniswap V2’s constant-product invariant ($K = 5000 \times 5000 = 25M$) keeps the pool balanced. Each 500 USDC swap shifts roughly 453 BUCK out of the pool. By month 12 the pool holds ~\$2,278 BUCK and ~\$11,000 USDC. Alice’s LP share captures all of this: she added \$5,000 of each, exits with \$2,232 BUCK and \$11,000 USDC.
- **Credit depreciation (bottom):** The on-chain `currentValue()` trace (thick red) follows the year-1 segment of the full 30-year schedule (dashed). The discrete whole-year compounding

formula produces a linear interpolation within each year, converging to \$8,576 at day 365. After 30 years the credit would reach its \$500 salvage floor.

2.5 Demurrage and Jubilee mechanics

During the year Alice holds no BUCK directly – it lives in the Uniswap V2 pair, which is bound as a Carrying account. Carrying accounts do not lose visible balance to demurrage; instead `_accrueJubilee` credits the Jubilee fund at the full `totalSupply` rate on every mint/burn.

When Alice removes liquidity, the pair executes a `_carryingTransfer` to her. The proportional `buckSeconds` for the BUCK she receives:

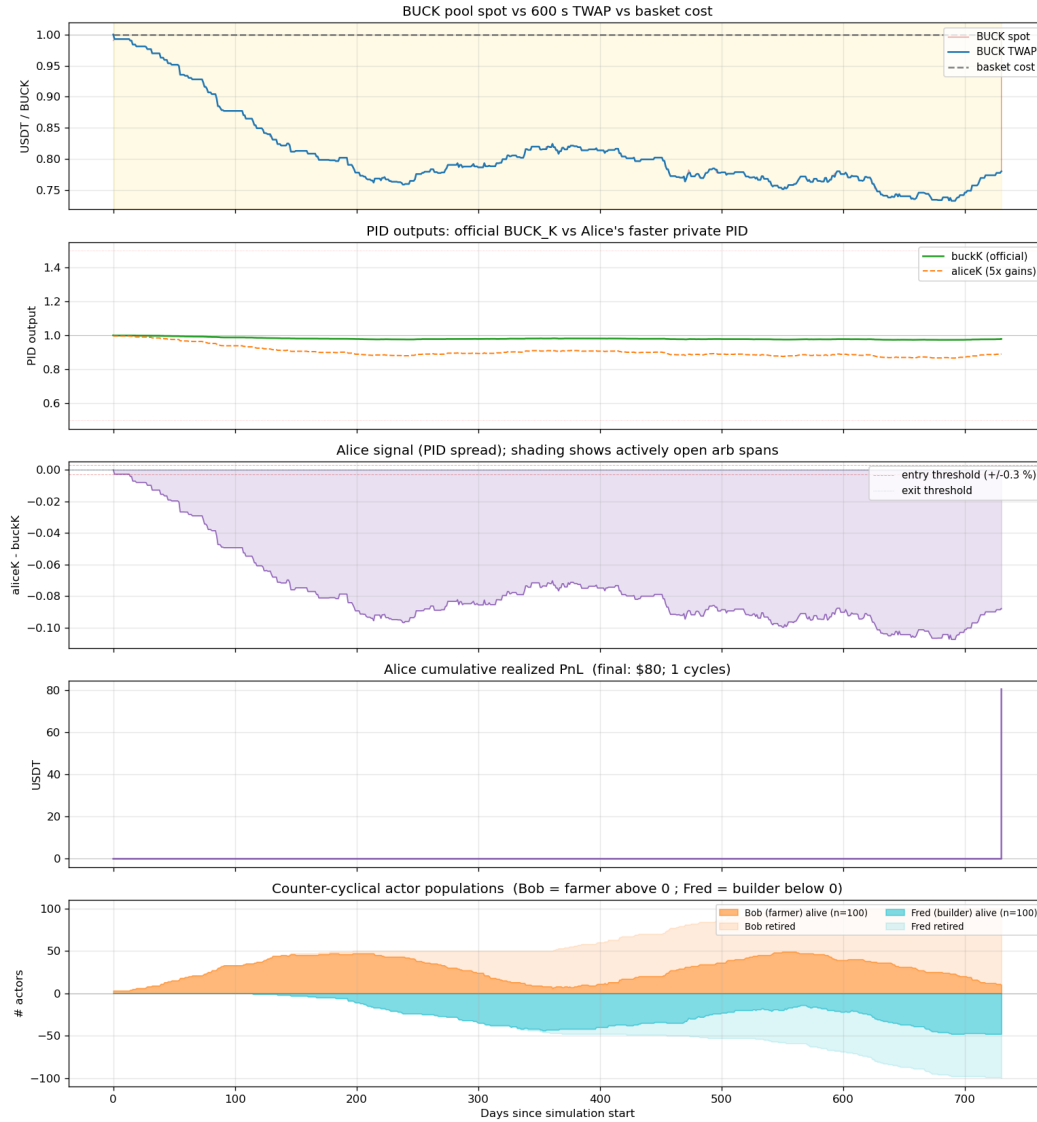
```
carried = liveBs * value / raw
```

arrive alongside the raw BUCK, so her spendable is \$2,232 while her raw balance is \$2,278 (the \$46 difference is the carried fee that locks into her slot).

The Jubilee accrues visibly only at the final `burn` call: $\$5,882 \times 2 \% \times 1 \text{ yr} = \117 . Governance can direct this accumulation to default-coverage, ground-rent disbursement, or a periodic balance reset – all on-chain paths land on the Jubilee slot.

3 BUCK Inflation/Deflation Arbitrage

This second example traces a market-making strategy around the `BUCK_K` controller. Alice provides deep BUCK/USDT liquidity AND keeps a BUCK + USDT reserve at home. As random Bobs mint, swap, and burn BUCK over a quarter, the BUCK/USDT pool spot drifts away from commodity- basket parity. The PID controller (reading a 600-second TWAP of the pool) eventually adjusts `BUCK_K` to push the system back to equilibrium – but the integration is slow by design and reads only the lagged TWAP. Alice trades against short-term excursions because she knows where parity lives and roughly when the PID will catch up.



3.1 Scenario

Parameter	Value
Alice's LP position	100,000 BUCK / 100,000 USDT (V3 0.3% fee tier)
Alice's wallet reserve	50,000 BUCK + 50,000 USDT (off-pool)
Basket composition	XAUT/USDT, PAXG/USDC, cbBTC/USDC, WBTC/USDT (25% each)
Basket cost (parity target)	\$1.00
BUCK_K initial / bounds	1.000 / [0.50, 1.50]
dT / dTMax / TWAP window	60 s / 3,600 s / 600 s
Time horizon	90 days, ~50 Bobs

Alice's edge:

- **On-chain visibility:** she watches `BuckKController.buckK()`, `lastBasketCost`, and her pool's `slot0` every block. She can forward-project where `buckK` will be one TWAP-window from now before the next `compute()` even runs.

- **Capital on both sides:** 50,000 BUCK and 50,000 USDT in reserve let her take either side of any drift without unwinding her LP first.
- **Patience:** her LP earns 0.3% on every Bob swap regardless of direction. Arbitrage is alpha on top of base swap-fee yield.

Risks she accepts:

- **Demurrage on her BUCK reserves:** her wallet is non-Carrying, so her off-pool BUCK accrues $\sim 2\%/yr$ in fee0wing. Her in-LP BUCK does not – the V3 pool is bound Carrying, so demurrage drips into the Jubilee at the totalSupply rate. Holding BUCK off-pool is a $2\%/yr$ cost.
- **PID overshoot:** buckK can clamp to a rail and basket re-weights shift the equilibrium. Positions can stay underwater for weeks.
- **Front-running:** every other arb sees the same on-chain signals. Alice’s edge is her commitment of capital before the PID-driven response fully materialises.

3.2 A typical round-trip

Trace one drift-and-arbitrage cycle:

1. **Day 12, 14:03.** Bob_17 calls `Buck.mint(8000e6)` and immediately swaps 8,000 BUCK for USDT in Alice’s pool. Pool moves to 108K BUCK / 92.6K USDT; spot = $92.6 / 108 = \mathbf{\$0.857}$. Alice collects ~ 24 USDT in 0.3% swap fees on her LP share.
2. **Day 12, 14:04.** `Buck.mint()` called `buckK.compute()`; `dT` had elapsed so the PID ran one cycle. But the PID reads the **600-second TWAP**: only ~ 2 s of post-swap history are in the window so far. TWAP = $\$0.998$, buckK barely moves: $1.0000 \rightarrow 1.0000$.
3. **Day 12, 14:30.** Several more Bobs cash out. Pool is now $\sim 109K / 91K$, spot $\$0.835$, TWAP caught up to $\sim \$0.842$. PID now sees a real $\sim -16\%$ error (BUCK - basket): buckK ratchets $1.0000 \rightarrow 0.995 \rightarrow 0.988 \rightarrow \dots \rightarrow 0.976$ over the next hour, contracting credit.
4. **Day 12, 14:35.** Alice judges the dislocation will revert (basket is stable, the PID is actively contracting buckK, which will force voluntary burns from over-extended holders – BUCK supply will shrink, lifting spot back toward parity). She swaps **20,000 USDT into BUCK** in her own pool, picking up $\sim 24,500$ BUCK at an effective rate of $\$0.816$ (V3 slippage included). Pool now 84.5K BUCK / 111K USDT. Her own swap snaps spot up to $\$1.31$, but the TWAP still averages near $\$0.85$ because most of the 600-second window predates her trade.
5. **Day 12, 15:00 - Day 14, 09:00.** PID continues lowering buckK (troughs ~ 0.96). The tighter credit limit prompts existing holders running near 100 % of limit to burn BUCK (buying BUCK from the pool with USDT, then burning it to release coverage capacity). The buy-back pressure plus the supply contraction pulls BUCK back toward basket from below. Other arbitrageurs see the same signal and pre-position long BUCK, accelerating the recovery.
6. **Day 14, 09:00.** Pool sits at 100K BUCK / 100.4K USDT, spot $\$1.004$, TWAP $\$1.001$. PID sees the error reverted. Over the next day buckK relaxes from 0.96 back toward 1.000 (the integral term has accumulated negative; recovery is gradual).

7. **Day 14, 09:30.** Alice swaps her 24,500 BUCK back for ~24,470 USDT at recovered parity, plus ~73 USDT of LP fees from other swaps during the 47-hour holding period. Round-trip net: +~**4,540 USDT** on a 20,000 USDT exposure.

The mirror-image trade applies in the over-valued direction: when a wave of Bob burns drains BUCK from the pool and pushes spot above basket, Alice sells BUCK from her reserve into the pool at a premium and buys back at recovered parity.

3.3 What the PID guarantees Alice

Alice's strategy works because the PID is, by design, slow:

- **TWAP smoothing:** a 10-minute TWAP attenuates a single-block manipulation by ~600x. It also lags real drift – TWAP doesn't reach 95% of a sustained spot move until ~3x window (30 minutes).
- **dT pacing:** even after `compute()` runs, only one PID cycle per 60 seconds. Across the 47-hour position above, ~2,800 cycles fired.
- **Anti-windup + bounds:** `buckK` saturates at [0.50, 1.50]. At peak drift it reached 1.040 – well within the rails.
- **dTMax clamp:** even if `compute()` goes silent for a day, the next call treats the gap as a single 1-hour step. Alice's strategy doesn't depend on continuous PID firing – `buckK` ratchets monotonically (in expectation) toward equilibrium regardless of cycle cadence.
- **dS compensation:** a basket-side shift (gold spike, governance re-weight) is subtracted out of the derivative term. Alice's position isn't whipsawed by oracle noise on the basket side.

The mechanism is **self-correcting in expectation**: when BUCK is undervalued, arbs like Alice buy BUCK with USDT, lifting spot. When BUCK is overvalued, arbs sell BUCK, depressing spot. In both directions arb action accelerates the PID's intended convergence.

Without arbitrageurs, `buckK` could swing without translating to spot-price recovery. With arbitrageurs, every basis point of `buckK` drift becomes capital flowing toward the under-priced side.

3.4 When the PID can't help her

Two scenarios break Alice's mean-reversion assumption:

- **Permanent basket re-weight:** governance changes the basket via `addBasketPool` or its inverse. `buckK` tracks the new basket cost; the old equilibrium evaporates. Alice should monitor governance events and unwind before any expected re-weighting.
- **Persistent oracle lag:** if the BUCK/USDT TWAP is much wider than the basket-pool TWAPs, the PID can persistently misread. The shipped configuration uses 600 s on every pool, so they're symmetric; asymmetric configurations create exploitable lag for the **operator** of whichever pool runs deeper.

3.5 Reproducing this example

A scripted Foundry test exercising this scenario lives at `test/BuckKArbitrageScenario.t.sol` (planned, see the Status section of alberta-buck-ethereum.org). It will deploy a V3 factory, all four basket pools at the reference prices, the BUCK/USDT pool with Alice's LP, populate the TWAP window, then run a scripted sequence of Bob mints + swaps + Alice arb trades over a simulated 90 days, emitting per-event state snapshots to `test/vectors/arb-scenario.json`. The Python visualisation (`alberta_buck/test/test_arb_plot.py`) turns that JSON into a four-panel chart: spot vs TWAP vs basket cost; buckK trajectory; Alice's PnL; aggregate Bob mint/burn flow.

4 Reproducing the Examples

```
# From the repo root inside nix develop:
forge test --match-test test_lifecycle           # generates test/vectors/lifecycle.json
python -m pytest alberta_buck/test/test_lifecycle_plot.py -v -s # regenerates images/lifecycle.
```

The lifecycle JSON is committed alongside the test so the plot can be regenerated without re-running the Forge suite. CI regenerates it and checks for drift with `git diff --exit-code`, the same discipline used for the SNARK test vectors.

The arbitrage scenario above is not yet wired into the test suite; when it lands, the same regenerate-then-diff discipline will apply.